ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC

CHỦ ĐỀ
XÂY DỰNG HỆ THỐNG PHÁT TRIỂN GIAN LẬN THẺ TÍN DỤNG THỜI GIAN THỰC

Giáo viên hướng dẫn: ThS. Phạm Minh Tú

Sinh viên thực hiện: 22120210 – Lê Võ Nhật Minh
22120227 – Nguyễn Hữu Nghĩa
22120262 – Nguyễn Lê Tấn Phát

Hồ Chí Minh, ngày 12 tháng 12 năm 2026

MỤC LỤC

Bảng phân công 5

I. TỔNG QUAN ĐỀ TÀI 6
1. Đặt vấn đề 6
a. Bối cảnh 6
b. Mục tiêu đề tài 6
2. Mô tả bài toán 6
a. Nguồn dữ liệu 6
b. Yêu cầu nghiệp vụ 7
3. Phạm vi đề tài 7

II. CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ 8
1. Apache Kafka 8
a. Giới thiệu 8
b. Vai trò trong hệ thống 8
2. Apache Spark & Spark Streaming 8
a. Giới thiệu 8
b. Lý do lựa chọn 8
3. HDFS và YARN 9
a. Giới thiệu 9
b. Vai trò trong hệ thống 9
4. Power BI 10
a. Giới thiệu 10
b. Vai trò trong hệ thống 10
5. Apache Airflow 10
a. Giới thiệu 10
b. Vai trò trong hệ thống 10

III. THIẾT KẾ HỆ THỐNG 11
1. Nút 1 (Node 1): Tầng thu thập dữ liệu (Ingestion Layer) 11
2. Nút 2 (Node 2): Tầng xử lý thời gian thực (Processing Layer) 12
3. Nút 3 (Node 3): Tầng quản lý và Lưu trữ (Management & Storage Layer) 12
4. Nút 4 (Node 4): Tầng phân tích và Báo cáo (Analytics & Serving Layer) 13

IV. Cài đặt và triển khai 15
1. Giai đoạn 1: Giả lập dữ liệu 15

a. Phát sinh dữ liệu giao dịch 15
b. Phát sinh tỉ giá 16
2. Giai đoạn 2: Xử lý thời gian thực 18
a. Module Đọc Tỉ Giá 18
b. Module Xử lý chính 18
3. Giai đoạn 3: Tối ưu hóa lưu trữ 20
a. Giải quyết vấn đề "Small File Problem" 20
b. Chiến lược phân vùng dữ liệu 20
4. Giai đoạn 4: Lên lịch 20
a. Lựa chọn công cụ: Apache Airflow 21
b. Quy trình vận hành của Pipeline 21
c. Ý nghĩa 21

V. Kết quả và phân tích 22
1. Câu 1 22
a. Thời điểm có lượng giao dịch cao nhất 22
b. Các khung giờ giao dịch bất thường 22
2. Câu 2 23
a. Thành phố có tổng giá trị giao dịch cao nhất 23
b. Liên hệ với vị trí và dân số 24
3. Câu 3 24
a. Merchant có giá trị giao dịch cao nhất 25
b. Merchant có số lượng giao dịch cao nhất 25
4. Câu 4 25
a. Các thành phố có tỷ lệ gian lận cao nhất 26
b. Các Merchant có tỷ lệ gian lận bất thường 26
5. Câu 5 26
a. Người dùng có nhiều giao dịch liên tiếp nhất 27
b. Phân tích khoảng cách thời gian giữa các giao dịch 27
6. Câu 6 27
a. Thời điểm thường xảy ra giao dịch giá trị lớn 28
b. Địa điểm thường xảy ra giao dịch giá trị lớn 28
7. Câu 7 29
a. Phân tích trọng điểm về Gian lận Trực tuyến 29
b. Xu hướng theo Thời gian 29
c. Xu hướng theo Đối tác 30
8. Câu 8 30
a. Quy mô và Lưu lượng giao dịch 31

b. Chỉ số tài chính (Financial Metrics) 31
c. Xu hướng hành vi theo khung giờ (Hourly Patterns) 31
9. Câu 9 31
a. Người dùng có số ca gian lận vượt mức trung bình 32
b. Phân tích phân bổ gian lận 32
10. Câu 10 32
a. Chiến lược giảm thiểu gian lận (Fraud Reduction) 33
b. Tối ưu hóa vận hành (Operational Optimization) 33
VI. TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN 34
1. Kết luận 34
2. Hạn chế 35
3. Hướng phát triển 35
VII. Tài liệu tham khảo 37

BẢNG PHÂN CÔNG

Bảng 1. Bảng phân công

STT	MSSV	HỌ VÀ TÊN	NHIỆM VỤ
1	22120210	Lê Võ Nhật Minh	- Triển khai mạng lưới Mesh VPN sử dụng Tailscale. - Triển khai HDFS, YARN - Triển khai Data Access Layer qua JDBC/ODBC cho Power BI. - Triển khai logic cho Kafka Producer
2	22120227	Nguyễn Hữu Nghĩa	- Xây dựng pipeline xử lý dữ liệu thời gian thực với Spark Structured Streaming. - Thiết kế module đọc tỉ giá đa luồng và làm sạch dữ liệu. - Tối ưu hóa chiến lược ghi dữ liệu xuống HDFS (Partitioning & Coalesce) để khắc phục vấn đề đường truyền mạng VPN.
3	22120262	Nguyễn Lê Tấn Phát	- Tạo dashboard và các biểu đồ trực quan hóa để trả lời các câu hỏi phân tích thực tế. - Tạo cổng ODBC để kết nối Spark với PowerBI Cloud và triển khai Azure Portal. - Cài đặt và thiết lập Apache Airflow đồng thời thiết kế lập lịch tự động.

I. TỔNG QUAN ĐỀ TÀI

1. Đặt vấn đề

a. Bối cảnh

Trong kỷ nguyên số hóa hiện nay, thanh toán không dùng tiền mặt đang trở thành xu hướng chủ đạo, đặc biệt là các giao dịch qua thẻ tín dụng tại các điểm bán hàng (POS). Cùng với sự bùng nổ về số lượng giao dịch, nguy cơ về gian lận tài chính cũng gia tăng nhanh chóng với các thủ đoạn ngày càng tinh vi.

Các hệ thống xử lý dữ liệu truyền thống theo lô (batch processing) thường có độ trễ lớn, không thể phát hiện và ngăn chặn các giao dịch gian lận ngay tại thời điểm phát sinh. Do đó, nhu cầu xây dựng một hệ thống xử lý dữ liệu lớn theo thời gian thực là vô cùng cấp thiết. Hệ thống này cần có khả năng tiếp nhận luồng dữ liệu liên tục từ các máy POS, phân tích tức thì để phát hiện lỗi hoặc gian lận, và cung cấp thông tin chi tiết cho việc ra quyết định kinh doanh.

b. Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một quy trình xử lý dữ liệu (End-to-End Data Pipeline) hoàn chỉnh cho một công ty tài chính, bao gồm các bước:

- Mô phỏng luồng dữ liệu giao dịch thẻ tín dụng phát sinh từ các máy POS theo thời gian thực.
- Xử lý, làm sạch và chuẩn hóa dữ liệu tức thì ngay khi giao dịch xảy ra.
- Tích hợp thông tin tỉ giá hối đoái để chuyển đổi tiền tệ sang VNĐ.
- Phát hiện và đánh dấu các giao dịch nghi ngờ gian lận hoặc bị lỗi.
- Lưu trữ dữ liệu tập trung và trực quan hóa các chỉ số thống kê quan trọng để hỗ trợ quản lý.

2. Mô tả bài toán

a. Nguồn dữ liệu

Dữ liệu đầu vào giả định là các giao dịch thẻ tín dụng được gửi về từ các máy POS tại các cửa hàng, nhà hàng, v.v..

- Dữ liệu được mô phỏng từ tập tin CSV có cấu trúc bao gồm các trường thông tin: thông tin người dùng (User), thẻ (Card), thời gian (Year, Month, Day, Time), số tiền

(Amount), thông tin đơn vị chấp nhận thẻ (Merchant Name, City, State), mã lỗi (Errors?) và nhãn gian lận (Is Fraud?).

- Hệ thống Kafka sẽ đóng vai trò Message Queue, đọc từng dòng dữ liệu CSV và gửi đến Topic để giả lập luồng giao dịch liên tục.

b. Yêu cầu nghiệp vụ

Hệ thống cần đáp ứng các yêu cầu xử lý cụ thể sau:

- Xử lý thời gian thực: Đọc dữ liệu từ Kafka ngay khi giao dịch phát sinh.
- Kiểm tra tính hợp lệ: Xác định giao dịch có bị lỗi hoặc gian lận hay không. Nếu Is Fraud = Yes, giao dịch được xem là lỗi/không thành công.
- Biến đổi dữ liệu:
  - Định dạng lại ngày giao dịch theo chuẩn dd/mm/yyyy.
  - Làm sạch số tiền (loại bỏ ký tự $ và lấy trị tuyệt đối).
  - Lấy tỉ giá USD/VND mới nhất từ nguồn bên ngoài (API/Web Scraping) để tính toán số tiền thực tế theo VNĐ.
- Lưu trữ: Dữ liệu sau khi xử lý thành công được lưu trữ tập trung vào hệ thống Hadoop (HDFS) để phục vụ thống kê lâu dài.
- Báo cáo & Trực quan hóa: Thống kê tổng giá trị giao dịch, số lượng giao dịch theo Merchant, phát hiện xu hướng gian lận theo thời gian/địa điểm và hiển thị trực quan trên Dashboard.

3. Phạm vi đề tài

Đề tài tập trung vào việc ứng dụng các công nghệ Big Data để giải quyết bài toán xử lý luồng dữ liệu. Các công nghệ cốt lõi được sử dụng bao gồm:

- Apache Kafka: Hệ thống trung gian (Message Broker) dùng để ingest dữ liệu và mô phỏng luồng sự kiện từ máy POS với độ trễ thấp.
- Apache Spark (Spark Streaming): Công cụ xử lý chính, thực hiện việc đọc dữ liệu từ Kafka, biến đổi (Transform), tính toán tỉ giá và làm sạch dữ liệu theo thời gian thực.
- Hadoop (HDFS): Hệ thống file phân tán dùng để lưu trữ bền vững (Data Lake) cho các dữ liệu đã qua xử lý.
- Power BI: Công cụ Business Intelligence dùng để kết nối với Hadoop, thực hiện các truy vấn phân tích và xây dựng Dashboard báo cáo trực quan.

- Apache Airflow: Công cụ lập lịch và quản lý quy trình (Orchestration), đảm bảo luồng dữ liệu được cập nhật và làm mới tự động theo chu kỳ.

II. CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ

1. Apache Kafka

a. Giới thiệu

Trong phạm vi dự án, Apache Kafka đóng vai trò là lớp trung chuyển dữ liệu trung tâm và hàng đợi thông điệp, đảm bảo tính toàn vẹn và hiệu năng cho đường ống xử lý dữ liệu.

b. Vai trò trong hệ thống

Cụ thể, Kafka được lựa chọn trong hệ thống vì:

- Đối với nguồn dữ liệu transactions, Kafka tiếp nhận luồng dữ liệu tốc độ cao được đẩy song song từ hệ thống đa tiến trình. Kafka hoạt động như một bộ đệm, giúp ổn định lưu lượng, ngăn chặn hiện tượng nghẽn cổ chai khi tốc độ sinh dữ liệu vượt quá khả năng xử lý của các hệ thống đích.
- Đối với dữ liệu exchange_rates, Kafka đóng vai trò điểm tích hợp thời gian thực.
- Hệ thống sử dụng cơ chế thu thập định kỳ từ API/Web bên ngoài và đẩy vào Kafka topic. Điều này giúp phân tách sự phụ thuộc trực tiếp giữa các module xử lý nội bộ và nguồn dữ liệu ngoại vi, đảm bảo tính sẵn sàng cao của hệ thống.
- Toàn bộ hệ thống chuyển sang mô hình hướng sự kiện, cho phép mở rộng linh hoạt và xử lý dữ liệu với độ trễ thấp thông qua cơ chế Publish/Subscribe của Kafka.

2. Apache Spark & Spark Streaming

a. Giới thiệu

Apache Spark là nền tảng xử lý dữ liệu phân tán mạnh mẽ, nổi bật với khả năng xử lý dữ liệu ngay trên RAM (In-memory processing). Trong đồ án này, nhóm sử dụng Spark Structured Streaming, module chuyên biệt để xử lý luồng dữ liệu thời gian thực dựa trên Spark SQL engine.

b. Lý do lựa chọn

Spark là lựa chọn tối ưu cho hệ thống phát hiện gian lận thẻ tín dụng vì:

- Tốc độ: Với cơ chế In-memory, Spark xử lý dữ liệu nhanh hơn MapReduce truyền thống gấp nhiều lần. Điều này đáp ứng yêu cầu "xử lý tức thì" (real-time).

- Cơ chế Micro-batch: Spark gom dữ liệu thành các lô nhỏ (batch) để xử lý. Điều này giúp cân bằng giữa tốc độ (latency thấp) và khả năng chịu tải cao (throughput), phù hợp với việc đọc liên tục từ Kafka.
- Hệ sinh thái: Spark hỗ trợ kết nối trực tiếp (native integration) với Kafka (đầu vào) và HDFS (đầu ra), giúp việc xây dựng pipeline liền mạch mà không cần cài thêm nhiều tool phụ trợ phức tạp.

3. HDFS và YARN

a. Giới thiệu

- HDFS là hệ thống tệp phân tán được thiết kế để chạy trên phần cứng thông thường, cung cấp khả năng lưu trữ dữ liệu với dung lượng lớn và khả năng chịu lỗi cao. HDFS chia dữ liệu thành các khối (blocks) – mặc định là 128MB – và phân tán chúng trên các node trong cụm.
- YARN là lớp quản lý tài nguyên của Hadoop, tách biệt chức năng quản lý tài nguyên khỏi chức năng lập lịch và giám sát công việc. Nó cho phép nhiều engine xử lý dữ liệu (như MapReduce, Spark) chạy đồng thời trên cùng một cluster.

b. Vai trò trong hệ thống

- HDFS đóng vai trò là Data Lake, chịu trách nhiệm lưu trữ an toàn các log giao dịch lớn. Ở phần này, nhóm sử dụng kiến trúc 2 Node:
  - Node 1 (Master - NameNode): Quản lý metadata (sổ địa chỉ của file), không chứa dữ liệu thật.
  - Node 2 (Worker - DataNode): Lưu trữ các block dữ liệu thực tế của giao dịch.
- YARN là hệ thống quản lý tài nguyên, chịu trách nhiệm phân bổ CPU và RAM cho Spark Executor chạy trên cluster, từ đó mở kết nối JDBC cho việc truyền tải dữ liệu đến Power BI. 2 node trong YARN có vai trò lần lượt là:
  - Node 1 (Master - ResourceManager): Tiếp nhận yêu cầu tính toán và cấp phát tài nguyên.
  - Node 2 (Worker - NodeManager): Quản lý tài nguyên tại chỗ để thực thi các tác vụ (Container) do Master giao xuống.

4. Power BI

a. Giới thiệu

Power BI là một công cụ Business Intelligence (BI) mạnh mẽ, được nhóm sử dụng để thực hiện chức năng báo cáo và trực quan hóa dữ liệu trong hệ thống. Đây là thành phần cuối cùng trong quy trình xử lý dữ liệu (End-to-End Data Pipeline), giúp chuyển đổi dữ liệu thô đã qua xử lý thành các thông tin có giá trị hỗ trợ ra quyết định.

b. Vai trò trong hệ thống

Trong phạm vi đề tài này, Power BI đóng vai trò kết nối trực tiếp với hệ thống lưu trữ phân tán Hadoop (HDFS). Nhiệm vụ chính của Power BI bao gồm:

- **Kết nối và Truy vấn:** Thiết lập kết nối đến Data Lake trên Hadoop và thực hiện các truy vấn phân tích cần thiết trên tập dữ liệu lớn.
- **Trực quan hóa (Visualization):** Xây dựng các Dashboard (bảng điều khiển) trực quan để hiển thị các chỉ số thống kê quan trọng phục vụ cho việc quản lý và giám sát.
- **Phân tích nghiệp vụ:** Cụ thể hóa các yêu cầu nghiệp vụ thông qua các biểu đồ, bao gồm:
    - Thống kê tổng giá trị giao dịch và số lượng giao dịch theo từng đơn vị chấp nhận thẻ (Merchant).
    - Phát hiện và hiển thị các xu hướng gian lận dựa trên các yếu tố thời gian và địa điểm.

Việc sử dụng Power BI giúp người quản trị có cái nhìn tổng quan và chi tiết về tình hình giao dịch, từ đó đưa ra các quyết định kinh doanh kịp thời dựa trên dữ liệu thời gian thực.

5. Apache Airflow

a. Giới thiệu

Apache Airflow là một nền tảng mã nguồn mở chuyên dụng để lập lịch, sắp xếp và giám sát các luồng công việc (workflows). Trong kiến trúc hệ thống dữ liệu lớn, đây là công cụ quản lý quy trình (Orchestration) đóng vai trò điều phối các tác vụ tự động.

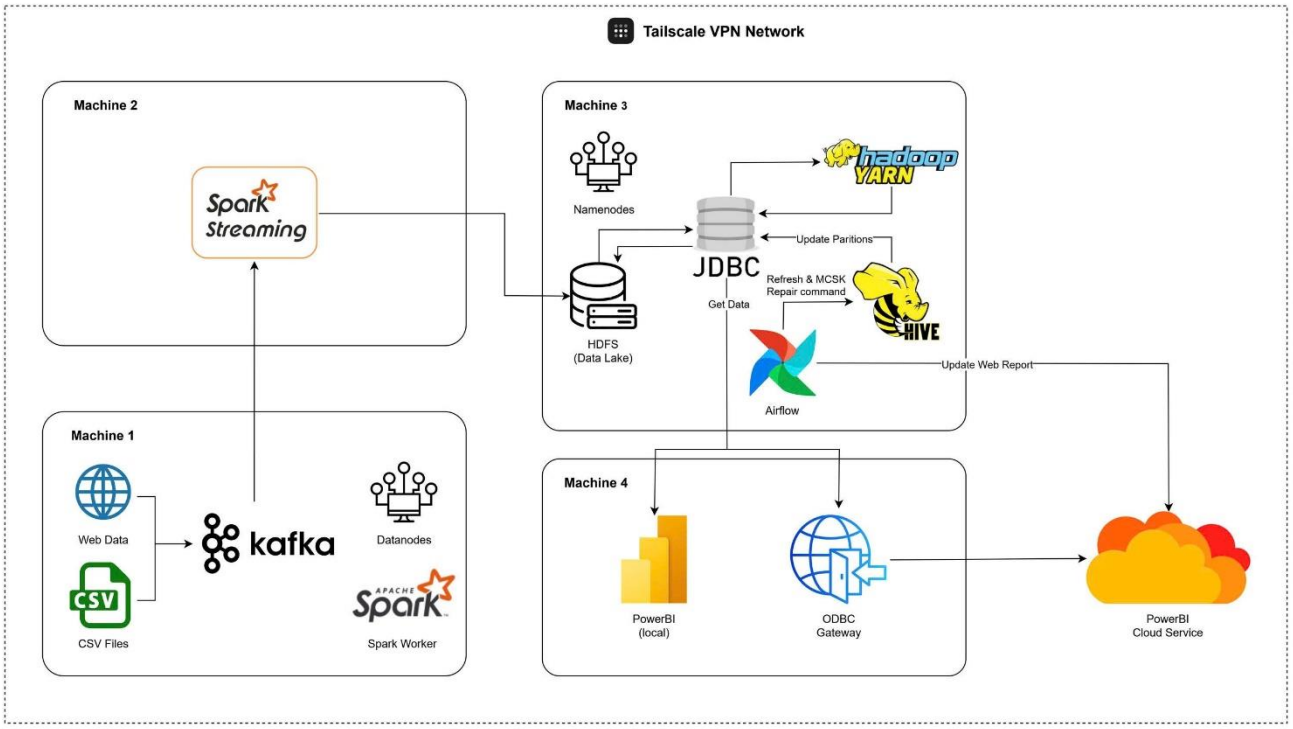
b. Vai trò trong hệ thống

Đối với hệ thống phát hiện gian lận thẻ tín dụng thời gian thực, Apache Airflow đảm nhận các chức năng quan trọng sau:

- **Lập lịch tự động (Scheduling):** Đảm bảo luồng dữ liệu trong toàn bộ hệ thống được cập nhật và làm mới tự động theo các chu kỳ định sẵn. Điều này tương ứng với "Giai đoạn 4: Lên lịch" trong quy trình triển khai hệ thống.
- **Quản lý quy trình (Orchestration):** Điều phối sự tương tác giữa các thành phần khác nhau của hệ thống, đảm bảo tính liên tục và ổn định của luồng dữ liệu từ khi tiếp nhận đến khi lưu trữ và báo cáo.

Sự hiện diện của Apache Airflow giúp hệ thống vận hành trơn tru, giảm thiểu sự can thiệp thủ công và đảm bảo dữ liệu mới nhất luôn sẵn sàng cho việc phân tích gian lận.

III. THIẾT KẾ HỆ THỐNG



- Hệ thống được triển khai theo mô hình Mini-Cluster gồm 04 nút tính toán (nodes), hoạt động trên môi trường mạng ảo hóa riêng (VPN) sử dụng giải pháp Tailscale. Kiến trúc này đảm bảo tính bảo mật và khả năng kết nối đồng nhất giữa các nền tảng hệ điều hành khác nhau (Windows, Linux, WSL).
- Mô hình xử lý dữ liệu tuân theo kiến trúc Lambda Architecture thu nhỏ, tập trung vào khả năng xử lý dòng dữ liệu (Streaming Processing) và lưu trữ phân tán (Distributed Storage).

1. **Nút 1 (Node 1): Tầng thu thập dữ liệu (Ingestion Layer)**

- Môi trường: Fedora Workstation 43

- Tài nguyên: 16GB RAM, 12 vCores (6 Physical Cores).
- Vai trò & Chức năng:
    - Data Ingestion (Kafka Producer): Thực hiện thu thập dữ liệu tỷ giá hối đoái từ Web (Web Scraping) và đọc dữ liệu mô phỏng từ tập tin .csv.
    - Message Broker (Kafka Server): Đóng vai trò là hàng đợi thông điệp trung gian, chuyển tiếp dữ liệu đến bộ xử lý Spark.
    - Distributed Processing (Worker): Chạy 01 DataNode (HDFS) và 01 Spark Worker để đóng góp tài nguyên tính toán và lưu trữ cho cụm.

2. **Nút 2 (Node 2): Tầng xử lý thời gian thực (Processing Layer)**

- Môi trường: Windows Subsystem for Linux (WSL).
- Tài nguyên: 16GB RAM, 12 vCores (6 Physical Cores).
- Vai trò & Chức năng:
    - Spark Streaming: Đảm nhận tác vụ tiêu thụ dữ liệu từ Kafka Producer tại Node 1. Thực hiện các phép biến đổi dữ liệu và ghi kết quả vào hệ thống tệp phân tán (HDFS).

3. **Nút 3 (Node 3): Tầng quản lý và Lưu trữ (Management & Storage Layer)**

- Instance: LXC - Debian 12 (Bookworm).
- Môi trường ảo hóa: Proxmox VE.
- Tài nguyên: 6GB RAM, 3 vCores.
- Vai trò & Chức năng (Master Node):
    - Cluster Management (NameNode & YARN): Quản lý không gian tên (Namespace) của HDFS và điều phối tài nguyên (Resource Manager) cho toàn bộ cụm.
    - Data Lake (HDFS): Lưu trữ dữ liệu thô và dữ liệu sau tiền xử lý từ Spark.
    - Data Serving (JDBC Interface): Cung cấp giao diện kết nối JDBC (thông qua Spark Thrift Server hoặc HiveServer2 trên nền YARN) phục vụ cho công cụ BI.
    - Orchestration (Apache Airflow):
        - Airflow Web Server: Cung cấp giao diện người dùng để quản lý, giám sát và kích hoạt các DAG (Directed Acyclic Graphs).

- Scheduler: Lập lịch trình dựa trên thời gian.
- Workers: Thực thi các tác vụ như gọi Spark Job (thông qua SSH), kiểm tra tính toàn vẹn của dữ liệu, và gửi lệnh làm mới sang Power BI.
- Azure Key Vault Integration: Airflow kết nối với Key Vault để lấy các thông tin nhạy cảm (Connection strings, API Keys của Power BI, Microsoft Credential) mà không cần lưu trực tiếp trong mã nguồn Python.
- Sau khi Task xử lý dữ liệu cuối cùng của Spark hoàn tất thành công.
- Airflow thực thi một PythonOperator sử dụng thư viện msal và requests để gửi lệnh POST đến Power BI REST API.
- Yêu cầu này kích hoạt quá trình Dataset Refresh, giúp báo cáo cập nhật số liệu gian lận mới nhất ngay lập tức.

4. **Nút 4 (Node 4): Tầng phân tích và Báo cáo (Analytics & Serving Layer)**

- Môi trường: Windows 11.
- Tài nguyên: 16GB RAM, 10 vCores (6 Physical Cores).
- Vai trò & Chức năng:
  - Business Intelligence (Power BI): Thiết kế và hiển thị báo cáo (Dashboard).
  - Connectivity (On-premises Data Gateway): Cầu nối bảo mật giữa hạ tầng Mini-Cluster (On-premise) và dịch vụ đám mây Azure (Power BI Service), cho phép làm mới dữ liệu từ cloud.
  - Để Power BI Web có thể "nhìn thấy" dữ liệu từ Spark (vốn nằm trong mạng nội bộ), hệ thống cần một kiến trúc kết nối bắc cầu thông qua Gateway và chuẩn kết nối ODBC.
    - ODBC (Open Database Connectivity) là giao diện tiêu chuẩn để Power BI kết nối với các engine xử lý dữ liệu lớn như Spark.
    - Lựa chọn Driver: Sử dụng Simba Spark ODBC Driver.
    - Thiết lập DSN (Data Source Name): Trên máy chủ cài đặt Gateway, thực hiện cấu hình trong ODBC Data Source Administrator:
    - Host: Địa chỉ IP/DNS của Thrift Server.
    - Port: 10000.

        - Mechanism: Chọn Username/Password .
        - Thrift Transport: Chọn HTTP.
    - Gateway đóng vai trò là một tác nhân (agent) nhận yêu cầu truy vấn từ Power BI Service (Cloud), thực thi nó dưới máy chủ cục bộ (Local) và trả kết quả ngược lại.
        - Cài đặt: Cài đặt bản Standard Mode.
        - Đăng ký Gateway: Đăng nhập bằng tài khoản Azure cùng Tenant với Power BI Service để đăng ký Gateway này lên Cloud.
        - Cấu hình trên Power BI Service (Web):
        - Truy cập Manage connections and gateways.
        - Tại Gateway đã cài đặt, thêm một Data Source mới.
        - Data Source Type: Chọn ODBC.
        - Connection String: Nhập chuỗi kết nối dựa trên DSN đã tạo (ví dụ: dsn=Spark_Fraud_Data).
        - Authentication: Nhập thông tin tài khoản truy cập vào Spark.
- Luồng hoạt động:
    - Azure Synapse Analytics / SQL Database: Sau khi Spark xử lý xong, dữ liệu được đẩy vào SQL Database để làm Serving Layer. Power BI sẽ kết nối với lớp này thay vì đọc trực tiếp từ HDFS để tối ưu tốc độ.
    - Power BI Service Principal: Sử dụng một Service Principal (App Registration) trong Azure AD để cấp quyền cho Airflow truy cập vào Power BI Workspace API.
    - Truy vấn qua Gateway: Power BI Service nhận lệnh REFRESH từ Airflow, gửi yêu cầu truy vấn thông qua Gateway.
    - Chuyển đổi ODBC: Gateway sử dụng Spark ODBC Driver để gửi lệnh SQL đến Spark Thrift Server.
    - Cập nhật: Dữ liệu kết quả được trả về, Power BI cập nhật các biểu đồ.

IV. CÀI ĐẶT VÀ TRIỂN KHAI

1. Giai đoạn 1: Giả lập dữ liệu

Ở phần này, nhóm lựa chọn tách module "Exchange Rate Producer" riêng biệt thay vì tích hợp vào Spark Streaming để tuân thủ kiến trúc Microservices và Event-driven, đảm bảo hệ thống ổn định và dễ mở rộng hơn và cũng bởi vì những lý do sau:

1. Tối ưu hiệu năng**:** Tránh hiện tượng nghẽn cổ chai trong Spark Streaming. Logic gọi API/Crawl Web là tác vụ I/O chậm, nếu tích hợp vào Spark sẽ làm tăng độ trễ xử lý, vi phạm nguyên tắc Real-time.
2. Tránh bị chặn truy cập**:** Spark là hệ thống phân tán với nhiều Worker chạy song song. Nếu tất cả Worker cùng gửi request đến website ngân hàng, hệ thống sẽ bị nhận diện là tấn công DDoS và bị chặn IP. Producer riêng biệt giúp kiểm soát tần suất gọi (5 phút/lần).
3. Tách biệt các mối lo**:** Spark chỉ nên tập trung vào xử lý tính toán dữ liệu lớn. Các tác vụ phụ trợ nặng nề và thiếu ổn định như chạy Headless Browser (sử dụng Playwright) nên được cô lập ở một service riêng để không tiêu tốn tài nguyên RAM/CPU của cụm Cluster.
4. Khả năng chịu lỗi cao**:** Nếu module lấy tỷ giá bị lỗi kết nối, Spark Streaming vẫn tiếp tục hoạt động bình thường (sử dụng tỷ giá cũ/mặc định trong Kafka hoặc Cache) thay vì làm sập toàn bộ luồng xử lý dữ liệu giao dịch.

a. Phát sinh dữ liệu giao dịch

Mã nguồn được thiết kế để giả lập quá trình đẩy dữ liệu giao dịch thẻ tín dụng theo thời gian thực vào hệ thống Kafka với khả năng xử lý song song và cơ chế chịu lỗi cơ bản.

a.1. Module tiền xử lý và phân chia dữ liệu

- Chức năng: Đảm bảo cân bằng tải cho các tiến trình con.
- Cài đặt chi tiết: Hàm split_csv_file()
  - Kiểm tra sự tồn tại của dữ liệu gốc (ORIGINAL_FILE).
  - Thực hiện chia nhỏ file dữ liệu lớn thành các phân đoạn (partitions) nhỏ hơn (part_1.csv, part_2.csv...) tương ứng với số lượng Worker được cấu hình (NUM_WORKERS = 3).
  - Kiểm tra GLOBAL_CHECKPOINT_FILE để xác định vị trí dữ liệu đã xử lý

trong lần chạy trước đó, tránh việc trùng lặp dữ liệu khi khởi động lại hệ thống chia file.

a.2. Module xử lý song song

- Chức năng: Đóng vai trò là các Producer độc lập, gửi dữ liệu vào Kafka Broker đồng thời để tăng thông lượng (Throughput).
- Công nghệ sử dụng: Thư viện multiprocessing của Python để tận dụng đa nhân CPU, vượt qua giới hạn của Global Interpreter Lock (GIL) so với kỹ thuật đa luồng (threading) thông thường.
- Cài đặt chi tiết: Hàm run_producer_worker(file_path, worker_id)
    - Đầu tiên, mỗi tiến trình khởi tạo một instance KafkaProducer riêng biệt để đảm bảo an toàn bộ nhớ (memory safety) và tránh xung đột kết nối.
    - Tiếp theo, dữ liệu giao dịch được tuần tự hóa thành định dạng JSON và mã hóa UTF-8 trước khi gửi.
    - Sử dụng time.sleep(random.uniform(1, 5)) để tạo độ trễ ngẫu nhiên giữa các lần gửi, mô phỏng hành vi giao dịch thực tế của người dùng thay vì gửi ồ ạt.
    - Mỗi worker duy trì một file checkpoint_{file_path}.txt riêng. Sau mỗi 50 bản ghi, vị trí hiện tại được lưu lại. Nếu worker bị crash và khởi động lại, nó sẽ đọc file này để tiếp tục gửi từ vị trí gián đoạn.

b. Phát sinh tỉ giá

Module producer_exchange_rates.py đảm nhận vai trò thu thập dữ liệu tỷ giá hối đoái (USD/VND) theo thời gian thực để phục vụ cho việc chuẩn hóa và quy đổi giá trị giao dịch trong hệ thống Big Data. Điểm đặc biệt của module này là việc áp dụng cơ chế Failover để đảm bảo dòng dữ liệu không bị gián đoạn ngay cả khi nguồn cung cấp chính gặp sự cố.

b.1. Module thu thập dữ liệu

Module này triển khai chiến lược thu thập bao gồm kênh chính và kênh dự phòng theo như yêu cầu từ đề bài:

- Kênh chính - API Consumer (get_exchange_rates_api):
    - Gửi request HTTP GET trực tiếp tới cổng thông tin XML của Vietcombank.
    - Sử dụng thư viện xml.etree.ElementTree để phân tích cú pháp XML, truy xuất thẻ Exrate có thuộc tính CurrencyCode='USD' và lấy giá trị chuyển khoản

(Transfer).

  - Tốc độ phản hồi nhanh, tiêu tốn ít tài nguyên hệ thống do không cần render giao diện web.
- Kênh dự phòng - Web Crawler (get_exchange_rates_crawler):
  - Chỉ được kích hoạt khi Kênh chính trả về lỗi hoặc không tìm thấy dữ liệu.
  - Sử dụng Playwright (đây là thư viện điều khiển trình duyệt tự động) kết hợp với asyncio để khởi tạo một trình duyệt Firefox không giao diện (headless browser). Từ đó giả lập hành vi người dùng truy cập trang chủ Vietcombank, chờ các script Dynamic Rendering tải hoàn tất, sau đó sử dụng BeautifulSoup để bóc tách dữ liệu từ bảng HTML.
  - Đây là phương án dự phòng để vượt qua các hạn chế về API (ví dụ: thay đổi endpoint, chặn request) từ đó đảm bảo luôn lấy được dữ liệu.

b.2. Module điều phối

Hàm coordinator() đóng vai trò là nhạc trưởng, quản lý logic nghiệp vụ và chu kỳ hoạt động của hệ thống:

- Thực hiện quy trình kiểm tra tuần tự:
  1. Gọi API (Kênh chính).
  2. Nếu API thất bại thì mới tiến hành gọi Crawler (Kênh phụ).
  3. Nếu cả hai thất bại thì ghi log lỗi và chờ chu kỳ sau (không làm sập chương trình).
- Sử dụng asyncio.sleep(300) để thiết lập chu kỳ thu thập dữ liệu tự động mỗi 5 phút (đây là giới hạn của API của VietcomBank). Sử dụng asyncio để không gây chặn (non-blocking) các tác vụ nền khác.

b.3. Module truyền tải dữ liệu

Sau khi thu thập thành công, dữ liệu được chuyển đến hàm send_to_kafka để đẩy vào đường ống xử lý:

- Dữ liệu được chuẩn hóa thành định dạng JSON bao gồm: loại tiền tệ (currency), tỷ giá (rate), và nhãn thời gian thực (timestamp chuẩn ISO-8601).
- Sử dụng KafkaProducer với cấu hình acks='all' để đảm bảo tính toàn vẹn dữ liệu (dữ liệu phải được ghi nhận trên tất cả các replica của Kafka trước khi coi là thành công).
- Tự động mã hóa dictionary của Python sang chuỗi JSON bytes trước khi gửi qua

mạng.

2. Giai đoạn 2: Xử lý thời gian thực

Giai đoạn này đảm nhận việc tiếp nhận dữ liệu thô từ Kafka, làm sạch, kết hợp với thông tin tỉ giá và lưu trữ xuống HDFS để phục vụ phân tích. Hệ thống được chia thành hai module hoạt động song song.

a. Module Đọc Tỉ Giá

Do tỉ giá hối đoái (USD/VND) biến động theo thời gian thực và độc lập với luồng giao dịch, nhóm đã thiết kế một cơ chế cập nhật bất đồng bộ sử dụng Multi-threading (Đa luồng).

- Cơ chế hoạt động: Hàm fetch_exchange_rate được chạy trên một luồng riêng biệt (Daemon Thread) song song với luồng xử lý chính của Spark. Luồng này sử dụng thư viện KafkaConsumer để liên tục lắng nghe topic exchange_rates.
- Cập nhật biến toàn cục: Khi có bản tin tỉ giá mới, luồng này sẽ cập nhật giá trị vào biến toàn cục CURRENT_USD_RATE. Nhờ đó, Spark Streaming khi xử lý các batch giao dịch luôn lấy được giá trị tỉ giá mới nhất mà không cần phải query lại database hay chờ đợi I/O, giúp giảm độ trễ xử lý.
- Kỹ thuật Consumer Group ngẫu nhiên: Trong code, nhóm sử dụng uuid để tạo group_id ngẫu nhiên cho mỗi lần khởi chạy (rate_reader_{uuid}). Điều này đảm bảo rằng consumer luôn được coi là mới và Kafka sẽ gửi ngay lập tức giá trị tỉ giá mới nhất (nhờ cơ chế *auto_offset_reset='earliest'* kết hợp với topic Compact), giúp hệ thống có dữ liệu tính toán ngay từ giây đầu tiên khởi động.

b. Module Xử lý chính

Đây là trái tim của hệ thống, sử dụng Spark Structured Streaming để xử lý dữ liệu theo cơ chế Micro-batch. Quy trình xử lý bao gồm các bước sau:

- Làm sạch và Chuẩn hóa dữ liệu (Data Cleaning)

Dữ liệu từ Kafka ở dạng JSON được parse (phân tích cú pháp) theo Schema định sẵn. Sau đó, Spark thực hiện các phép biến đổi:

  - Xử lý số tiền (Amount): Dữ liệu thô từ hệ thống nguồn (Kafka) được gửi dưới dạng chuỗi ký tự có chứa định dạng tiền tệ (ví dụ: $150.5). Nhóm sử dụng hàm regexp_replace để loại bỏ ký tự $ dư thừa, sau đó thực hiện ép kiểu dữ liệu về

dạng số thực (DoubleType). Bước chuẩn hóa này là bắt buộc để hệ thống có thể thực hiện các phép tính toán học (nhân với tỉ giá, tính tổng) ở các bước tiếp theo.

- Định dạng ngày tháng: Các trường rời rạc Day, Month, Year được ghép lại thành chuỗi duy nhất theo định dạng chuẩn dd/MM/yyyy bằng hàm concat, giúp thuận tiện cho việc truy vấn thời gian sau này.

- Chiến lược Xử lý Gian lận

Một điểm nhấn quan trọng trong thiết kế hệ thống là cách xử lý các giao dịch được đánh dấu là gian lận (Is Fraud? = "Yes"). Thay vì lọc bỏ (filter) các dòng này khỏi luồng dữ liệu, nhóm quyết định giữ lại toàn bộ và tạo thêm cột cờ hiệu (Flag).

- Kỹ thuật: Sử dụng hàm when().otherwise() để tạo cột Is_Fraud_Flag:
  - Nếu Is Fraud? là "Yes" → Giá trị = 1.
  - Nếu Is Fraud? là "No" → Giá trị = 0.
- Lý do giữ lại dữ liệu: Việc giữ lại các giao dịch gian lận là cốt lõi để phục vụ các bài toán phân tích thống kê trong tương lai (cụ thể là các yêu cầu số 4, 7 và 9 của đề tài):
  - Tính tổng thiệt hại: Cho phép tính tổng tiền (SUM(Amount)) của các giao dịch gian lận.
  - Phân tích xu hướng: So sánh tỉ lệ giữa giao dịch sạch và giao dịch gian lận theo thời gian.
  - Vẽ bản đồ rủi ro: Xác định các Merchant hoặc khu vực địa lý có tần suất xuất hiện Is_Fraud_Flag = 1 cao nhất.

- Chuẩn bị ghi dữ liệu

Trước khi dữ liệu được lưu trữ lâu dài, hệ thống thực hiện bước kiểm soát cuối cùng:

- Deduplication (Loại bỏ trùng lặp): Sử dụng hàm dropDuplicates dựa trên bộ khóa định danh (User, Time, Amount, Merchant) ngay trong từng micro-batch. Điều này đảm bảo nguyên tắc Exactly-once, loại bỏ hoàn toàn các bản tin bị gửi lặp (duplicate messages) do cơ chế mạng của Kafka, đảm bảo tính chính xác cho báo cáo tài chính.
- Chuẩn bị ghi: Dữ liệu sau khi làm sạch sẽ được chuyển sang giai đoạn tối ưu hóa lưu trữ (sẽ được trình bày chi tiết ở mục 3) để giải quyết các vấn đề về hiệu năng

HDFS.

3. **Giai đoạn 3: Tối ưu hóa lưu trữ**

Sau khi xử lý logic, thách thức lớn nhất là làm sao lưu trữ dữ liệu hiệu quả trên HDFS để vừa ghi nhanh vừa đọc nhanh. Nhóm đã giải quyết hai bài toán kỹ thuật sau:

a. **Giải quyết vấn đề "Small File Problem"**

- Vấn đề: Do Spark Streaming xử lý dữ liệu theo các batch nhỏ (Micro-batch), nếu ghi trực tiếp xuống HDFS với nhiều Worker chạy song song, hệ thống sẽ tạo ra hàng ngàn file kích thước rất nhỏ (vài KB). Điều này gây áp lực lớn lên NameNode (nơi quản lý metadata của HDFS) và làm giảm hiệu suất nghiêm trọng.
- Giải pháp - Kỹ thuật Coalesce(1): Trong mã nguồn, nhóm sử dụng phương thức .coalesce(1) trước khi ghi. Lệnh này buộc Spark gom toàn bộ dữ liệu của các phân vùng trong bộ nhớ về một Worker duy nhất.
    - Kết quả: Mỗi batch chỉ tạo ra đúng 1 file vật lý duy nhất. Kỹ thuật này không chỉ khắc phục vấn đề file nhỏ mà còn giảm tải tối đa kết nối mạng khi hệ thống vận hành qua môi trường mạng VPN/Tailscale hạn chế băng thông.

b. **Chiến lược phân vùng dữ liệu**

Để tối ưu hóa tốc độ truy vấn cho các báo cáo Power BI, dữ liệu không được đổ dồn vào một thư mục chung mà được tổ chức theo cấu trúc tầng bậc (Hierarchical Directory Structure).

- Cấu trúc: Sử dụng lệnh .partitionBy("p_Year", "p_Month", "p_Day").
- Đường dẫn vật lý: */user/project/processed_data/p_Year=2025/p_Month=01/p_Day=09/...csv*
- Hiệu quả: Khi cần truy vấn dữ liệu của một ngày cụ thể, Spark/Hive chỉ cần quét đúng thư mục con tương ứng và bỏ qua toàn bộ dữ liệu của các ngày khác (*cơ chế Partition Pruning*), giúp giảm thời gian truy vấn từ hàng phút xuống vài giây.

4. **Giai đoạn 4: Lên lịch**

Mục tiêu của giai đoạn này là đảm bảo luồng dữ liệu từ HDFS, sau khi được xử lý và tối ưu hóa, sẽ được cập nhật lên hệ thống báo cáo Power BI một cách nhất quán và kịp thời mà không cần sự can thiệp thủ công.

a. Lựa chọn công cụ: Apache Airflow

Chúng ta sử dụng Apache Airflow làm nền tảng điều phối (Orchestrator). Airflow cho phép định nghĩa quy trình dưới dạng DAG (Directed Acyclic Graph), giúp quản lý các tác vụ theo trình tự, xử lý lỗi và lên lịch chạy chính xác.

b. Quy trình vận hành của Pipeline

Pipeline được thiết kế để chạy định kỳ mỗi 10 phút, bao gồm hai bước chính:

- Cập nhật Metadata (Hive Sync):
  - Sử dụng BashOperator để gọi Beeline.
  - Lệnh MSCK REPAIR TABLE: Giúp Hive nhận diện các phân vùng (partitions) mới vừa được tạo ra ở Giai đoạn 3 trong HDFS.
  - Lệnh REFRESH TABLE: Xóa bộ nhớ đệm cũ để đảm bảo các truy vấn sau đó lấy được dữ liệu mới nhất.
- Kích hoạt làm mới Power BI (Power BI Trigger):
  - Sử dụng PythonOperator kết hợp với thư viện msal (Microsoft Authentication Library).
  - Hệ thống sẽ thực hiện xác thực OAuth2 dưới danh nghĩa người dùng để lấy access_token.
  - Gửi một yêu cầu POST tới Power BI REST API để yêu cầu Dataset làm mới dữ liệu từ nguồn Hive/Hadoop.

c. Ý nghĩa

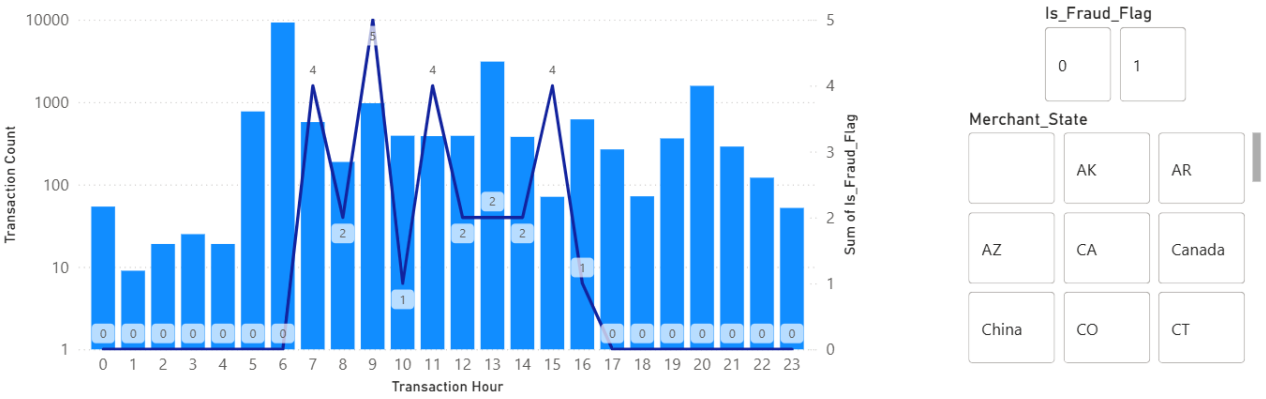
- Tính toàn vẹn: Đảm bảo Power BI không bao giờ hiển thị dữ liệu cũ trong khi dữ liệu mới đã nằm trong kho (Data Warehouse).
- Tính tự động: Giảm thiểu rủi ro từ việc thao tác tay, cho phép hệ thống vận hành 24/7.
- Khả năng mở rộng: Dễ dàng thêm các bước kiểm tra chất lượng dữ liệu (Data Quality Check) vào giữa quy trình trước khi đẩy lên báo cáo.

V. KẾT QUẢ VÀ PHÂN TÍCH

1. Câu 1

Câu hỏi: Thời điểm nào trong ngày có nhiều giao dịch nhất? Có khung giờ nào giao dịch bất thường không?

Transaction Day of the Week	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	Total
Friday	9	2	2	5		98	1351	66	27	131	46	53	59	436	48	7	87	45	9	53	224	54	20	3	**2835**
Monday	6	2	2	5	4	103	1302	85	33	159	64	53	65	422	54	19	108	47	14	43	224	41	20	11	**2886**
Saturday	12	2	1	10	5	115	1330	99	21	141	59	45	69	459	46	8	89	28	10	45	231	32	20	7	**2884**
Sunday	11	1	5		2	120	1344	76	33	132	49	70	66	443	51	8	81	33	12	63	219	41	15	5	**2880**
Thursday	8			2	3	108	1284	90	24	140	62	58	41	449	71	11	78	42	9	48	220	44	14	14	**2820**
Tuesday	4	1	4		1	114	1316	79	27	134	47	46	48	457	56	11	88	39	8	39	221	35	11	5	**2791**
Wednesday	4	1	5	3	4	113	1328	78	23	133	64	62	42	438	53	7	87	33	10	72	237	42	21	7	**2867**
Total	**54**	**9**	**19**	**25**	**19**	**771**	**9255**	**573**	**188**	**970**	**391**	**387**	**390**	**3104**	**379**	**71**	**618**	**267**	**72**	**363**	**1576**	**289**	**121**	**52**	**19963**



a. Thời điểm có lượng giao dịch cao nhất

Dựa vào biểu đồ cột (Transaction Count), biểu đồ sử dụng thang đo Logarit (1, 10, 100, 1000...) để hiển thị rõ sự chênh lệch cực lớn về lưu lượng giữa các khung giờ mà vẫn giữ được tính trực quan cho các khung giờ thấp điểm và bảng tổng hợp, chúng ta xác định được:

- Khung giờ cao điểm: 6 giờ sáng (Hour 6).
- Lượng giao dịch: Đạt xấp xỉ 9.3K (9,300 giao dịch).

Đây là thời điểm có sự gia tăng đột biến về số lượng. So với các khung giờ khác (thường dưới 2K), lượng giao dịch tại khung 6 giờ sáng cao gấp khoảng 4-5 lần mức trung bình.

Nhìn vào bảng phía trên, lượng giao dịch lớn vào khung giờ 6 này xuất hiện đều đặn ở tất cả các ngày trong tuần (từ Thứ Hai đến Chủ Nhật đều ghi nhận trên 1,200 giao dịch/ngày tại cột số 6).

b. Các khung giờ giao dịch bất thường

Có hai dấu hiệu bất thường đáng chú ý cần đưa vào báo cáo để cảnh báo hoặc điều tra thêm:

- Bất thường về lưu lượng tại khung 6 giờ sáng:

Hiện tượng: Lượng giao dịch vọt lên cực cao (9.3K) nhưng ngay sau đó (giờ 7) và trước đó (giờ 5) lại thấp hơn rất nhiều.

Đây có thể không phải là giao dịch từ người dùng thực tế (vì 6 giờ sáng là giờ thấp điểm tiêu dùng). Có khả năng đây là các giao dịch tự động (Automated/Scheduled payments) hoặc lệnh

quét thanh toán định kỳ của hệ thống.

- Bất thường về gian lận tại khung 7 giờ sáng:

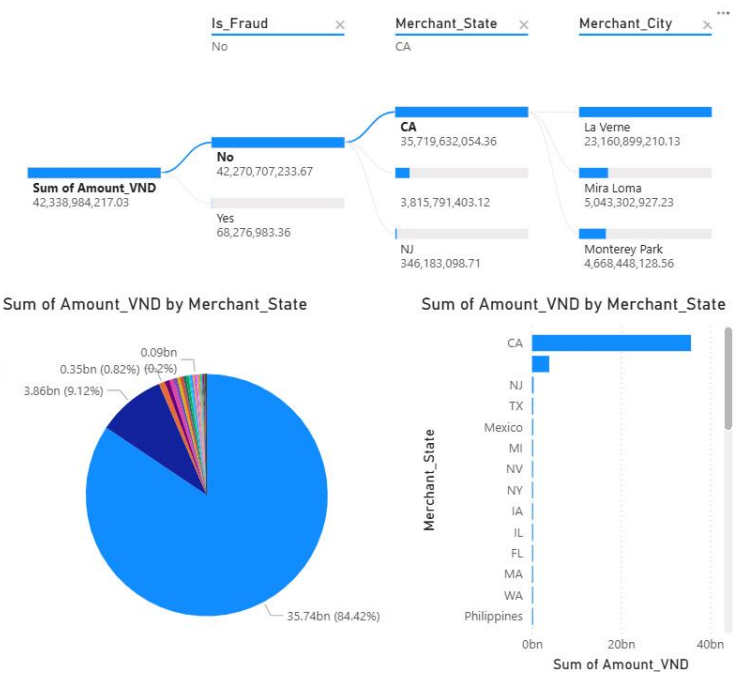
Hiện tượng: Mặc dù khung giờ 6 có lượng giao dịch lớn nhất nhưng số ca gian lận (Sum of Is_Fraud_Flag) lại bằng 0. Tuy nhiên, ngay sau đó vào 7 giờ sáng (Hour 7), lượng giao dịch giảm mạnh nhưng số ca gian lận lại đạt cao (4 ca).

Khung giờ từ 7h - 9h và 11h, 15h là những khoảng thời gian nhạy cảm, nơi tỷ lệ gian lận trên tổng giao dịch cao hơn hẳn các khung giờ khác.

Hệ thống ghi nhận sự tách biệt rõ rệt giữa "Lưu lượng lớn" và "Rủi ro lớn". Các khung giờ sáng sớm (6h) mang lại lưu lượng, nhưng các khung giờ hành chính (7h - 11h) và đầu giờ chiều (15h) mới là thời điểm tội phạm gian lận hoạt động mạnh.

2. Câu 2

Câu hỏi: Thành phố nào có tổng giá trị giao dịch cao nhất? Có liên hệ với dân số hoặc vị trí không?



a. Thành phố có tổng giá trị giao dịch cao nhất

Dựa trên biểu đồ cây phân tách và dữ liệu chi tiết từ báo cáo:

- Thành phố dẫn đầu: La Verne.
- Tổng giá trị giao dịch: Đạt mức kỷ lục hơn 23,16 tỷ VND (23,160,899,210.13 VND).
- Vị trí: Thành phố này thuộc bang California (CA), Hoa Kỳ.

Giá trị giao dịch tại La Verne cao gấp hơn 4,5 lần so với thành phố đứng thứ hai trong cùng bang là Mira Loma (5,04 tỷ VND). Điều này cho thấy một sự tập trung dòng tiền cực lớn vào một điểm merchant cụ thể.

b. Liên hệ với vị trí và dân số

- Mối liên hệ với vị trí địa lý (Location)

Biểu đồ hình tròn và biểu đồ cột cho thấy bang California (CA) chiếm tỷ trọng áp đảo với 35,74 tỷ VND, tương đương 84,42% tổng giá trị giao dịch của toàn bộ hệ thống.

Bản đồ nhiệt cho thấy các điểm giao dịch phát sinh chủ yếu tại Bắc Mỹ (đặc biệt là bờ Tây) và Châu Âu. Điều này khẳng định giao dịch tập trung tại các khu vực có hạ tầng tài chính và công nghệ phát triển nhất.

- Mối liên hệ với dân số

Có một sự liên hệ mật thiết giữa quy mô dân số và giá trị giao dịch ở cấp bang. California là bang đông dân nhất Hoa Kỳ, do đó việc bang này chiếm hơn 84% giá trị giao dịch là điều dễ hiểu.

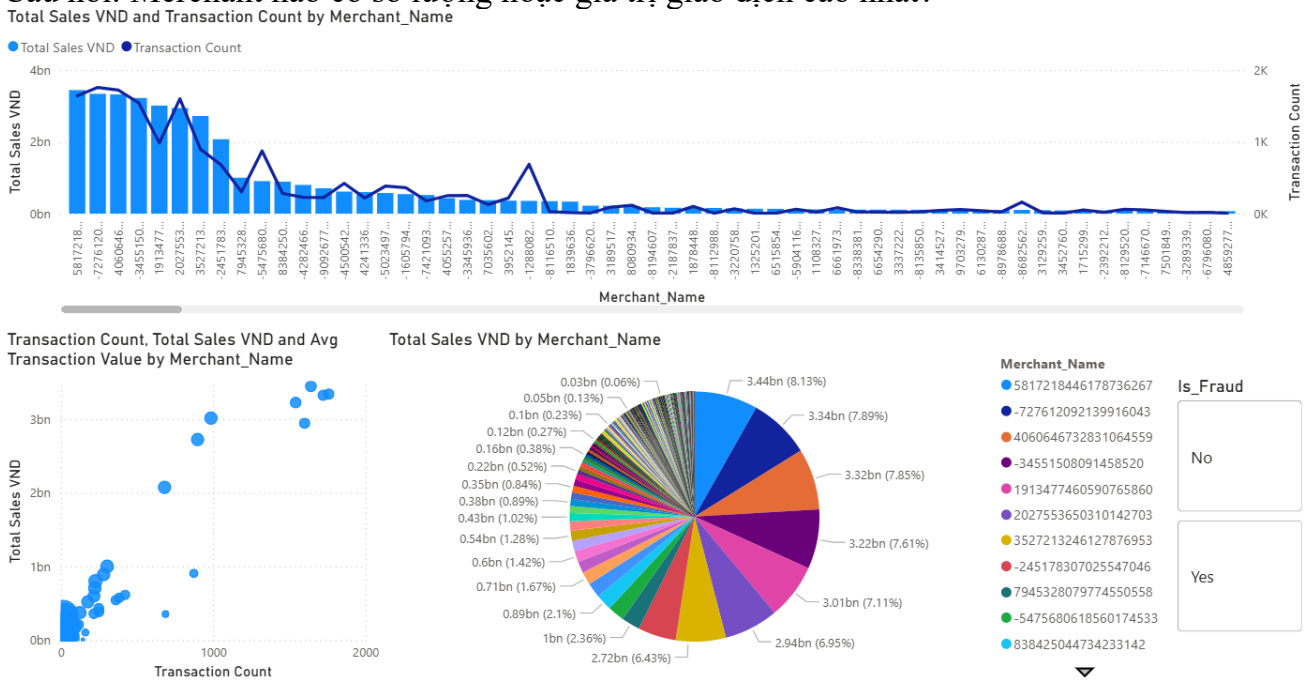
Tuy nhiên, ở cấp độ thành phố, La Verne không phải là thành phố có dân số lớn nhất tại California (so với các đại đô thị như Los Angeles hay San Diego).

Việc một thành phố có quy mô dân số vừa phải như La Verne dẫn đầu về giá trị giao dịch (23,16 tỷ VND) cho thấy dòng tiền không chỉ phụ thuộc vào số lượng cư dân, mà phụ thuộc vào vị trí đặt các trung tâm thương mại lớn, trụ sở doanh nghiệp hoặc các cổng thanh toán tập trung.

La Verne (CA) là trung tâm giao dịch lớn nhất, đóng góp hơn 54% tổng giá trị giao dịch toàn hệ thống. Bang California vẫn là thị trường cốt yếu cần tập trung ưu tiên (chiếm 84,42% giá trị).

3. Câu 3

Câu hỏi: Merchant nào có số lượng hoặc giá trị giao dịch cao nhất?



a. Merchant có giá trị giao dịch cao nhất

Dựa trên Pie Chart và Bar Chart về tổng doanh thu, chúng ta xác định được các đối tác dẫn đầu:

- Merchant đứng đầu: Mã đối tác 5817218446178736267.
- Tổng giá trị giao dịch: Đạt 3,44 tỷ VND, chiếm tỷ trọng 8.13% trên tổng doanh số toàn hệ thống.
- Các đối tác xếp sau:
  - Mã -727612092139916043 với 3,34 tỷ VND (7.89%).
  - Mã 4060646732831064559 với 3,32 tỷ VND (7.85%).

b. Merchant có số lượng giao dịch cao nhất

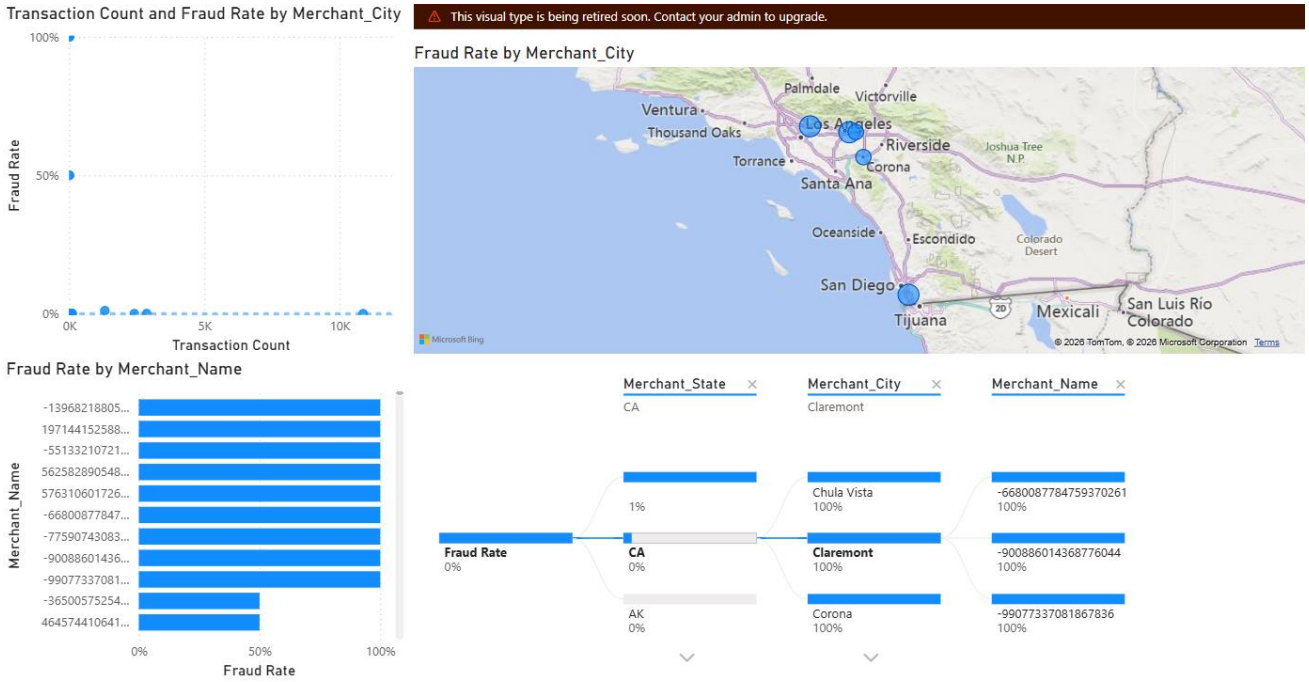
Quan sát biểu đồ kết hợp và biểu đồ phân tán:

- Đối tác dẫn đầu: Merchant 5817218446178736267 cũng đồng thời là đơn vị có số lượng giao dịch cao nhất.
- Lưu lượng: Số lượng giao dịch của Merchant này tiệm cận mức 2.000 giao dịch (đỉnh cao nhất của đường kẻ màu xanh đậm trên biểu đồ).
- Nhận xét về mối tương quan: Biểu đồ phân tán ở góc trái bên dưới cho thấy một đường thẳng đi lên rõ rệt. Điều này khẳng định rằng đối với các Merchant này, số lượng giao dịch và tổng giá trị giao dịch có tỉ lệ thuận rất chặt chẽ (số lượng giao dịch càng nhiều thì doanh thu càng cao).

Top 5 Merchant hàng đầu đóng góp hơn 35% tổng giá trị giao dịch của hệ thống. Đây là nhóm khách hàng/đối tác chiến lược cần có chính sách ưu đãi và quản lý rủi ro riêng biệt. Các Merchant có giá trị giao dịch cao nhất không chỉ có số lượng giao dịch lớn mà còn có giá trị trung bình trên mỗi giao dịch ổn định.

4. Câu 4

Câu hỏi: Thành phố hoặc merchant nào có tỷ lệ fraud cao bất thường?



a. Các thành phố có tỷ lệ gian lận cao nhất

Thông qua biểu đồ cây phân tách và bản đồ nhiệt về tỷ lệ gian lận, chúng ta nhận diện được 3 "điểm đen" tại bang California (CA):

- Thành phố có tỷ lệ rủi ro 100%:
  - Claremont
  - Chula Vista
  - Corona
- Theo bản đồ "Fraud Rate by Merchant_City", các khu vực gian lận tập trung mạnh tại cụm phía Nam California (khu vực xung quanh Los Angeles và San Diego). Đây là các khu vực cần được đưa vào danh sách giám sát đặc biệt.

b. Các Merchant có tỷ lệ gian lận bất thường

Biểu đồ cột "Fraud Rate by Merchant_Name" và biểu đồ phân tán cho thấy một thực trạng đáng báo động:

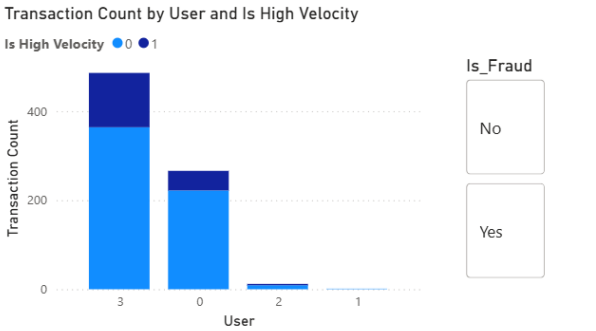
- Danh sách Merchant rủi ro tuyệt đối (100% Fraud Rate): Nhiều mã đối tác như -13968218805..., 197144152588..., -55133210721... ghi nhận tỷ lệ gian lận là 100%.
- Dựa trên biểu đồ Transaction Count and Fraud Rate, các điểm có tỷ lệ gian lận 100% và 50% đều nằm sát trục tung (số lượng giao dịch rất thấp, gần mức 0).
- Đây là hành vi điển hình của việc thử thẻ hoặc các tài khoản bị đánh cắp thực hiện một vài giao dịch thử nghiệm rồi bị khóa ngay lập tức. Các Merchant này không đóng góp vào doanh thu thực tế nhưng lại gây ra rủi ro hệ thống lớn.

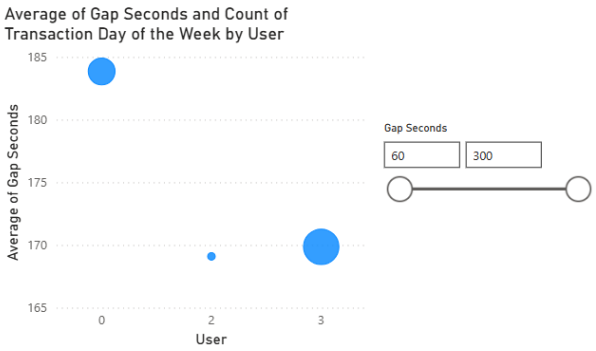
Tỷ lệ gian lận 100% tập trung chủ yếu ở các Merchant có lưu lượng giao dịch cực thấp. Và gian lận phân bổ theo cụm địa lý rõ rệt tại 3 thành phố: Claremont, Chula Vista và Corona.

5. Câu 5

Câu hỏi: Người dùng nào có nhiều giao dịch liên tiếp trong thời gian ngắn?

User	Year	Quarter	Month	Day	Average of Amount_VND	Average of Gap Seconds
0	2002	Qtr 3	September	11	897,392.80	300.00
0	2002	Qtr 4	November	23	3,367,049.09	300.00
3	2003	Qtr 1	January	24	1,786,959.50	300.00
0	2003	Qtr 3	August	13	1,742,611.60	300.00
3	2004	Qtr 1	March	6	-1,486,959.00	300.00
3	2004	Qtr 1	March	16	1,524,524.28	300.00
3	2004	Qtr 3	July	16	3,464,092.73	300.00
0	2004	Qtr 3	August	10	973,305.97	300.00
3	2004	Qtr 3	September	27	129,391.52	300.00
3	2004	Qtr 4	November	15	1,782,524.71	300.00
0	2005	Qtr 1	January	16	3,459,397.07	300.00
3	2005	Qtr 2	May	10	0.00	300.00
0	2005	Qtr 2	June	5	777,392.60	300.00
0	2005	Qtr 3	July	26	-2,349,003.92	300.00
3	2005	Qtr 3	August	23	-2,400,004.00	300.00
0	2005	Qtr 4	November	17	800,349.16	300.00
0	2005	Qtr 4	November	29	1,176,262.83	300.00
0	2006	Qtr 1	January	7	978,262.50	300.00
0	2006	Qtr 1	January	27	3,692,875.72	300.00
3	2007	Qtr 1	January	5	350,609.28	300.00
3	2007	Qtr 1	January	11	1,408,698.00	300.00
0	2007	Qtr 1	March	2	3,873,919.50	300.00
0	2007	Qtr 2	April	18	1,156,697.58	300.00
3	2007	Qtr 2	April	25	1,533,132.99	300.00
Total					**1,197,336.69**	**174.72**





a. Người dùng có nhiều giao dịch liên tiếp nhất

Dựa trên biểu đồ "Transaction Count by User and Is High Velocity", chúng ta xác định được đối tượng trọng tâm:

- Người dùng số 3: Là người có tổng số lượng giao dịch cao nhất hệ thống, đạt mức gần 500 giao dịch.
- Đặc điểm High Velocity: User 3 cũng sở hữu lượng giao dịch bị gắn cờ "Is High Velocity = 1" (giao dịch tần suất cao trong thời gian ngắn) lớn nhất so với các người dùng khác.
- Người dùng xếp sau: User 0 đứng thứ hai về tổng lượng giao dịch, nhưng tỷ lệ giao dịch tần suất cao thấp hơn đáng kể so với User 3.

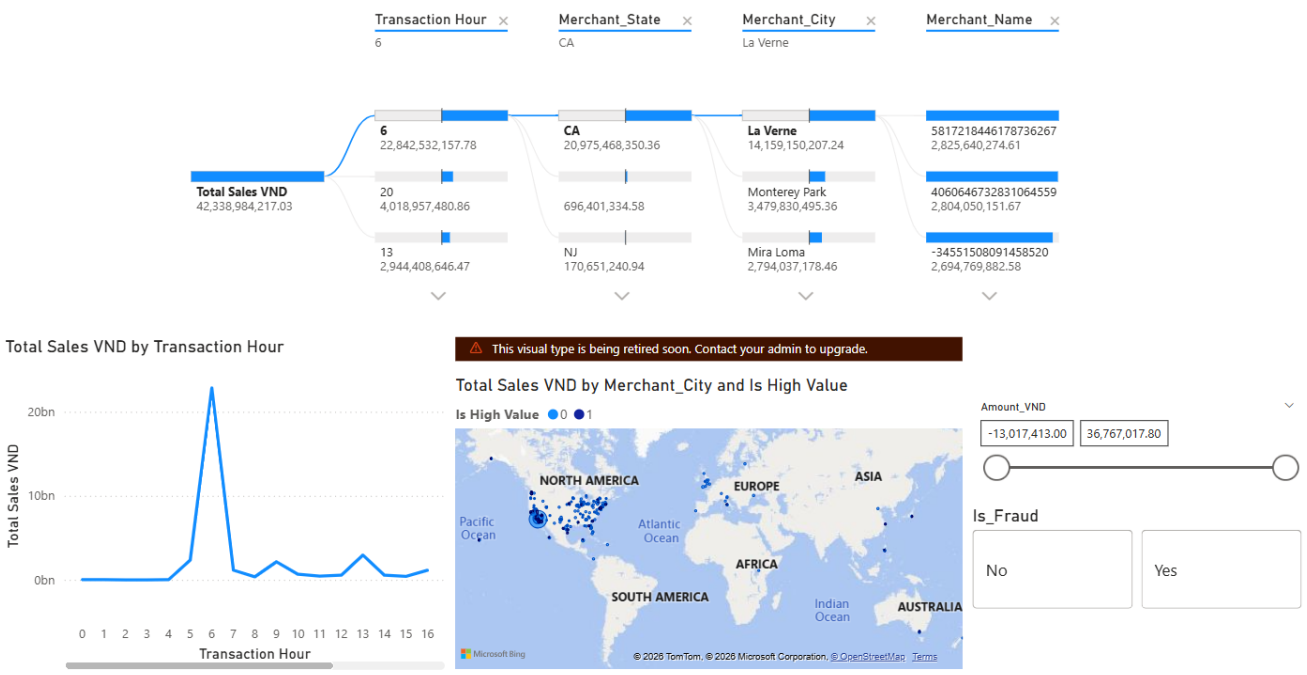
b. Phân tích khoảng cách thời gian giữa các giao dịch

Dựa trên biểu đồ phân tán và bảng dữ liệu chi tiết, chúng ta thấy rõ tốc độ giao dịch của các đối tượng này:

- User 3 (Tốc độ nhanh nhất): Có khoảng cách thời gian trung bình giữa các giao dịch thấp nhất, chỉ khoảng 170 giây (chưa tới 3 phút mỗi giao dịch). Kích thước vòng tròn lớn trên biểu đồ cho thấy số ngày phát sinh giao dịch của người này cũng rất dày đặc.
- User 0: Có khoảng cách trung bình dài hơn, xấp xỉ 184 giây.
- Điểm bất thường trong bảng dữ liệu: Bảng chi tiết ghi nhận nhiều chuỗi giao dịch của User 0 và User 3 có khoảng cách thời gian cố định chính xác là 300.00 giây (đúng 5 phút). Đây là dấu hiệu điển hình của việc sử dụng công cụ thanh toán tự động (Bot/Scripts) thay vì thao tác của người dùng thực.

6. Câu 6

Câu hỏi: Giao dịch có giá trị lớn thường xảy ra vào thời điểm nào? Ở đâu?



a. Thời điểm thường xảy ra giao dịch giá trị lớn

Dựa vào biểu đồ "Total Sales VND by Transaction Hour" và sơ đồ cây phân tách doanh thu:

- Khung giờ đỉnh điểm: Giao dịch giá trị lớn tập trung áp đảo vào lúc 6 giờ.
- Giá trị cụ thể: Chỉ riêng tại khung 6 giờ sáng, tổng giá trị giao dịch đã đạt hơn 22,84 tỷ VND, chiếm hơn một nửa tổng doanh thu toàn hệ thống (42,34 tỷ VND).
- Các khung giờ đáng chú ý khác: Doanh thu có xu hướng tăng nhẹ trở lại vào lúc 20 giờ (khoảng 4,02 tỷ VND) và 13 giờ (khoảng 2,94 tỷ VND).

b. Địa điểm thường xảy ra giao dịch giá trị lớn

Sử dụng tính năng phân tách để truy xuất nguồn gốc dòng tiền lớn từ khung giờ 6:00 sáng:

- Khu vực trọng điểm (Bang): Bang California (CA) là nơi phát sinh phần lớn giá trị giao dịch, đóng góp 20,98 tỷ VND (tương đương gần 92% doanh thu của khung giờ 6:00).
- Thành phố dẫn đầu: Tại California, thành phố La Verne là "điểm nóng" nhất với giá trị giao dịch lên tới 14,16 tỷ VND.
- Các thành phố khác: Ngoài La Verne, các thành phố như Monterey Park (3,48 tỷ VND) và Mira Loma (2,79 tỷ VND) cũng ghi nhận lượng tiền giao dịch lớn trong cùng khung giờ.
- Phân bổ địa lý toàn cầu: Bản đồ thế giới cho thấy các giao dịch giá trị lớn (High Value) tập trung dày đặc nhất tại khu vực Bắc Mỹ (North America), đặc biệt là vùng ven biển phía Tây thuộc Hoa Kỳ.

Giao dịch giá trị lớn không phân bổ đều mà tập trung cực kỳ đậm đặc vào một khoảng thời gian hẹp (6:00 sáng) và một địa điểm cụ thể (La Verne, California). Đặc biệt Merchant mã số

5817218446178736267 tại La Verne chính là đơn vị tạo ra doanh thu lớn nhất (hơn 2,8 tỷ VND) trong khung giờ này.

7. Câu 7

Câu hỏi: Có xu hướng nào trong các giao dịch bị fraud không? (giờ, merchant, city,...)

Day of the Week	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	Total
Friday	0	0	0	0		0	0	2	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3
Monday	0	0	0	0	0	0	0	2	2	4	0	2	0	1	0	0	0	0	0	0	0	0	0	0	11
Saturday	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Sunday	0	0	0		0	0	0	0	0	0	0	2	2	1	0	2	0	0	0	0	0	0	0	0	7
Thursday	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Tuesday	0	0	0		0	0	0	0	0	0	1	0	0	0	2	2	1	0	0	0	0	0	0	0	6
Wednesday	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Total	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**4**	**2**	**5**	**1**	**4**	**2**	**2**	**2**	**4**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**27**

a. Phân tích trọng điểm về Gian lận Trực tuyến

Dựa trên sơ đồ cây phân tách tổng số ca gian lận, ta thấy rằng các giao dịch trực tuyến chiếm số lượng lớn nhất:

- Kênh rủi ro nhất: Giao dịch ONLINE chiếm tỷ trọng lớn nhất với 15/27 ca gian lận (chiếm 55,5% tổng số ca).
- Đối tượng cần giám sát: Trong kênh Online, Merchant mã số -8566951830324093739 là đơn vị đáng báo động nhất với 3 ca gian lận riêng biệt.

Ta cũng thấy có một sự tập trung rõ rệt về mặt địa lý khi mà các khu vực bờ Tây Hoa Kỳ lại chiếm tỷ trọng khá lớn:

- Bang trọng điểm: California (CA) là nơi tập trung gian lận lớn nhất với 12 ca (chiếm gần 45% tổng số ca gian lận toàn hệ thống).
- Thành phố "điểm đen":
  - Các thành phố như Claremont, Chula Vista, và Corona ghi nhận tỷ lệ gian lận tuyệt đối là 100% (mọi giao dịch phát sinh đều là gian lận).
  - Xét về số lượng tuyệt đối, Corona và Upland dẫn đầu tại bang CA với mỗi thành phố có 3 ca gian lận.

b. Xu hướng theo Thời gian

Dữ liệu cho thấy gian lận không xảy ra ngẫu nhiên mà tập trung vào các khung giờ cụ thể trong ngày:

- Ngày rủi ro nhất: Thứ Hai (Monday) là ngày bùng phát gian lận mạnh nhất với 11 ca. Ngược lại, Thứ Tư, Thứ Năm và Thứ Bảy hoàn toàn không ghi nhận ca gian lận nào.
- Khung giờ "nóng":
  - Gian lận tập trung chủ yếu vào ban ngày, từ 7 giờ sáng đến 15 giờ chiều.
  - Đỉnh điểm: Lúc 9 giờ sáng ghi nhận số ca gian lận cao nhất trong ngày (5 ca). Riêng ngày Thứ Hai, khung 9h sáng đã có tới 4 ca.
  - Các khung giờ 7h, 11h và 15h cũng ghi nhận mức rủi ro cao với 4 ca/giờ.

c. Xu hướng theo Đối tác

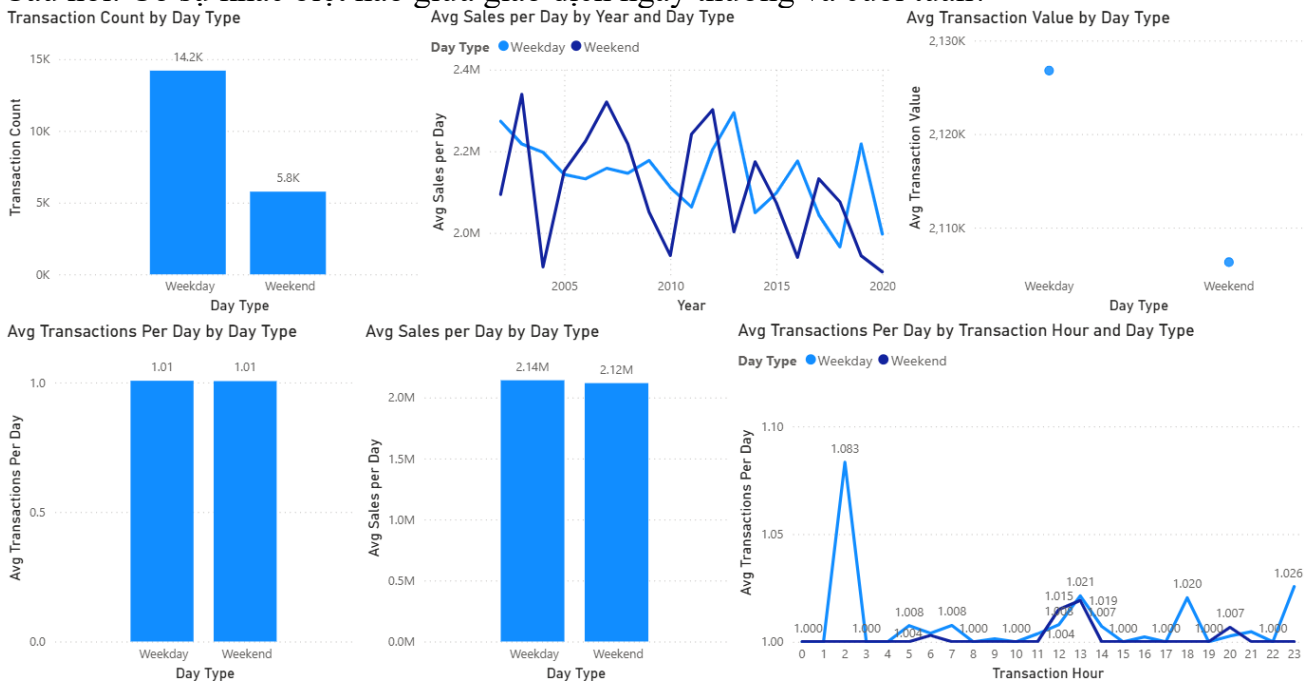
Có sự khác biệt lớn giữa các Merchant uy tín và Merchant gian lận:

- Tập trung vào giá trị nhỏ: Biểu đồ phân tán cho thấy các giao dịch có tỷ lệ gian lận cao (50% - 100%) hầu hết đều có tổng giá trị giao dịch rất thấp, tiệm cận mức 0 tỷ VND.
- Giao dịch lớn an toàn hơn: Các giao dịch có giá trị lớn (từ 2 tỷ đến 4 tỷ VND) đều nằm ở mức 0% Fraud Rate (Is_Fraud = No). Điều này cho thấy kẻ gian thường thực hiện các giao dịch nhỏ để tránh bị hệ thống kiểm soát phát hiện ngay lập tức.

Một giao dịch có nguy cơ gian lận cao nhất khi phát sinh từ bang California, tại các thành phố như Claremont hoặc Corona, vào khung giờ từ 9h sáng hoặc 15h chiều. Đồng thời các giao dịch giá trị nhỏ liên tục tại các Merchant mới hoặc Merchant có lưu lượng thấp cũng là dấu hiệu cảnh báo sớm của các đợt tấn công gian lận.

8. Câu 8

Câu hỏi: Có sự khác biệt nào giữa giao dịch ngày thường và cuối tuần?



a. Quy mô và Lưu lượng giao dịch

Ngày thường chiếm ưu thế tuyệt đối với khoảng 14,2K giao dịch, cao gấp gần 2,5 lần so với cuối tuần (chỉ khoảng 5,8K giao dịch).
Tuy tổng số lượng khác nhau, nhưng số lượng giao dịch trung bình phát sinh mỗi ngày (Avg Transactions Per Day) của cả hai nhóm đều xấp xỉ mức 1.01.

b. Chỉ số tài chính

Giá trị giao dịch trung bình (Avg Transaction Value): Ngày thường có giá trị trung bình cao hơn, đạt khoảng 2.127K VND, trong khi cuối tuần thấp hơn một chút, ở mức 2.105K VND.
Doanh số trung bình mỗi ngày (Avg Sales per Day): Ngày thường ghi nhận mức doanh thu trung bình là 2,14M VND, nhỉnh hơn so với mức 2,12M VND của cuối tuần.

c. Xu hướng hành vi theo khung giờ

Đây là điểm khác biệt thú vị nhất về hành vi của người dùng:

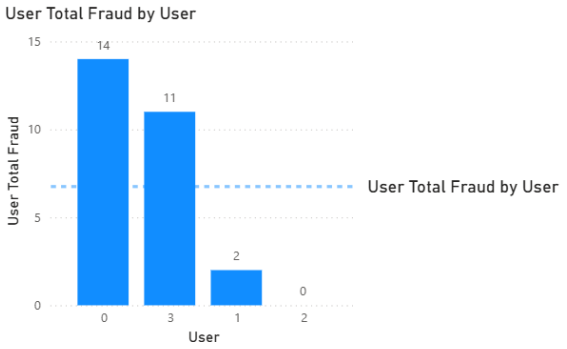
- Ngày thường: Ghi nhận một sự đột biến cực lớn vào lúc 2 giờ sáng với tỷ lệ giao dịch trung bình đạt 1.083. Điều này có thể liên quan đến các quy trình thanh toán tự động hoặc hành vi đặc thù của nhóm khách hàng làm việc đêm.

Cuối tuần: Giao dịch có xu hướng rải rác và tập trung vào các khung giờ sinh hoạt giải trí như 13 giờ (1.021), 18 giờ (1.020) và đặc biệt tăng mạnh vào cuối ngày lúc 23 giờ (1.026).

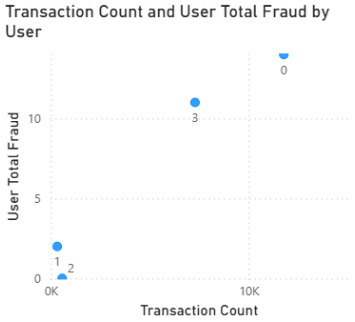
Ta có thể kết luận được rằng, ngày thường là thời điểm mang lại nguồn thu chính và lưu lượng giao dịch lớn nhất cho hệ thống. Tuy nhiên, đây cũng là lúc tội phạm gian lận hoạt động mạnh nhất, đặc biệt là vào đầu tuần (Thứ Hai).

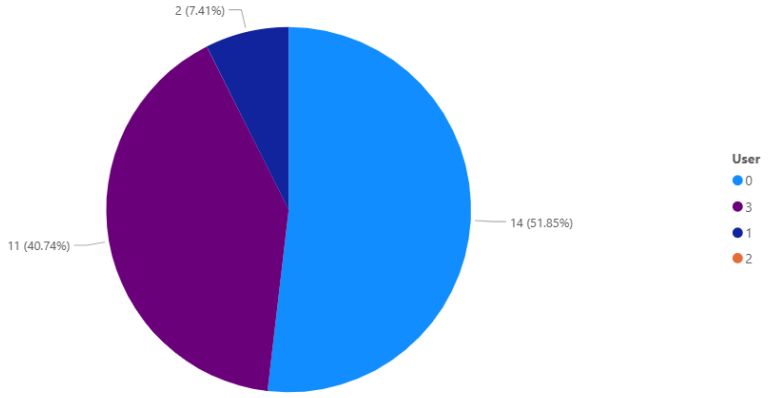
9. Câu 9

Câu hỏi: Có người dùng nào bị nhiều lỗi hoặc bị gắn cờ fraud nhiều hơn mức trung bình?



User	User Total Fraud	Fraud Ratio vs Average
0	14	0.52
1	2	0.07
2	0	0.00
3	11	0.41





a. Người dùng có số ca gian lận vượt mức trung bình

Dựa trên biểu đồ User Total Fraud by User, hệ thống ghi nhận tổng cộng 27 ca gian lận chia cho 4 người dùng chính, với mức trung bình xấp xỉ 6.75 ca/người.

Có hai người dùng nằm trong nhóm "nguy cơ cao" vì có số ca gian lận vượt xa mức trung bình này:

- User 0: Dẫn đầu danh sách với 14 ca gian lận. Người dùng này chiếm tới 51.85% tổng số ca gian lận toàn hệ thống.
- User 3: Đứng thứ hai với 11 ca gian lận, chiếm tỷ trọng 40.74%.

Ngược lại, User 1 (2 ca) và User 2 (0 ca) đều nằm dưới mức rủi ro trung bình.

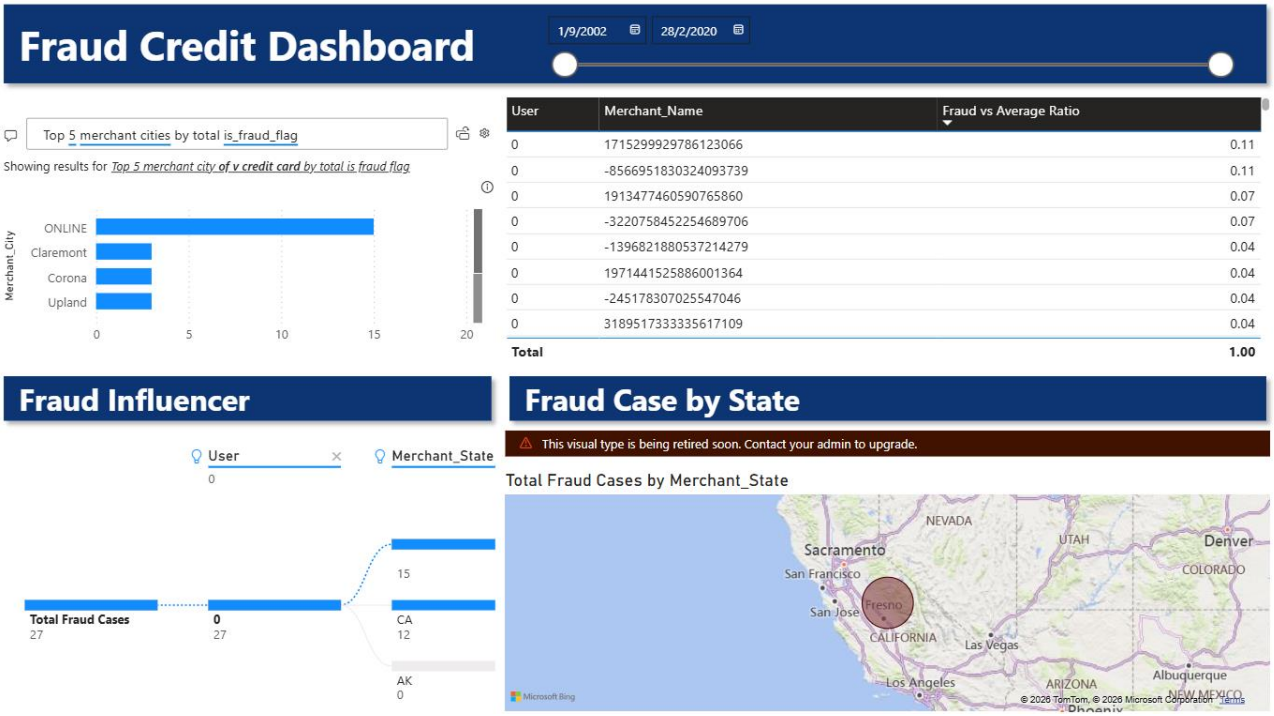
b. Phân tích phân bổ gian lận

Biểu đồ tròn minh họa sự tập trung rủi ro cực lớn vào một nhóm nhỏ người dùng:

- Phần lớn tổng số ca gian lận (hơn 92%) chỉ tập trung vào User 0 và User 3.
- User 1 chỉ chiếm một phần nhỏ (7,41%), và User 2 hoàn toàn không ghi nhận ca gian lận nào.

10. Câu 10

Câu hỏi: Từ các phân tích trên, hãy đề xuất cải tiến cho hệ thống để giảm gian lận hoặc tối ưu vận hành.



a. Chiến lược giảm thiểu gian lận (Fraud Reduction)

Dựa trên các "điểm nóng" rủi ro đã nhận diện, hệ thống cần triển khai các lớp bảo mật tự động:

- Kiểm soát theo địa điểm rủi ro: Thiết lập cơ chế tự động yêu cầu xác thực đa yếu tố (MFA) hoặc tạm dừng giao dịch để kiểm tra thủ công đối với các yêu cầu phát sinh từ các thành phố có tỷ lệ gian lận 100% như Claremont, Chula Vista và Corona.
- Giám sát đối tượng trọng điểm: Thực hiện khóa tạm thời hoặc rà soát chuyên sâu tài khoản của User 0 và User 3, vì hai đối tượng này chiếm tới 92,59% tổng số ca gian lận toàn hệ thống.
- Phát hiện và chặn Bot giao dịch: Triển khai thuật toán nhận diện "tần suất lặp lại cố định". Hệ thống nên gắn cờ cảnh báo rủi ro cao cho bất kỳ chuỗi giao dịch nào có khoảng cách thời gian lặp lại chính xác (ví dụ: đúng 300,00 giây) để chặn đứng các hành vi giao dịch tự động của bot.
- Tăng cường bảo mật kênh Online: Do kênh trực tuyến chiếm đa số ca gian lận (15/27 ca), cần áp dụng các tiêu chuẩn bảo mật thanh toán khắt khe hơn (như 3D Secure 2.0) cho các giao dịch Online, đặc biệt là vào Thứ Hai — ngày có số ca gian lận cao nhất.

b. Tối ưu hóa vận hành (Operational Optimization)

Dựa trên xu hướng lưu lượng và giá trị giao dịch để phân bổ nguồn lực hợp lý:

- Quản lý tải hệ thống theo giờ cao điểm: Hệ thống máy chủ và đội ngũ hỗ trợ kỹ thuật cần được ưu tiên sẵn sàng vào lúc 6 giờ sáng, thời điểm ghi nhận lưu lượng giao dịch đột biến lên tới 9.255 giao dịch.
- Điều phối nhân sự theo loại ngày: Vì số lượng giao dịch ngày thường (14,2K) cao gấp gần 2,5 lần cuối tuần (5,8K), doanh nghiệp nên tập trung đội ngũ vận hành và chăm sóc khách hàng vào các ngày trong tuần để đảm bảo hiệu suất xử lý.
- Ưu tiên đối tác chiến lược: Tập trung hạ tầng ổn định và các chương trình ưu đãi cho các Merchant tại thành phố La Verne (CA), vì đây là nơi mang lại giá trị giao dịch lớn nhất (hơn 23 tỷ VND).
- Cảnh báo theo dõi khung giờ rủi ro: Thiết lập hệ thống giám sát thời gian thực tăng cường trong khung giờ từ 7 giờ sáng đến 15 giờ chiều, tập trung vào các giao dịch giá trị nhỏ (dấu hiệu thử thẻ) để ngăn chặn kịp thời trước khi gian lận quy mô lớn xảy ra.

VI. TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đề tài đã hoàn thành mục tiêu xây dựng một quy trình xử lý dữ liệu (End-to-End Data Pipeline) hoàn chỉnh cho bài toán phát hiện gian lận thẻ tín dụng theo thời gian thực. Các kết quả chính đạt được bao gồm:

- **Kiến trúc hệ thống:** Đã triển khai thành công mô hình Mini-Cluster gồm 04 nút tính toán hoạt động trên mạng ảo hóa Tailscale VPN, kết hợp linh hoạt giữa các môi trường Windows, WSL và Linux Server.
- **Công nghệ tích hợp:** Hệ thống đã kết nối và vận hành trơn tru các công nghệ Big Data cốt lõi: Apache Kafka cho việc điều phối luồng dữ liệu , Apache Spark Streaming cho xử lý in-memory tốc độ cao , và HDFS/YARN cho lưu trữ phân tán.
- **Xử lý nghiệp vụ:** Giải quyết tốt các bài toán kỹ thuật phức tạp như cập nhật tỷ giá hối đoái đa luồng (Multi-threading) , xử lý vấn đề "Small File Problem" trong lưu trữ HDFS , và cơ chế phân vùng dữ liệu để tối ưu truy vấn.
- **Trực quan hóa và Phân tích:** Thông qua Power BI, hệ thống đã cung cấp các Dashboard trực quan, trả lời được 10 câu hỏi nghiệp vụ quan trọng về xu hướng gian lận, hành vi người dùng (User 0, User 3) và các điểm nóng rủi ro về địa lý

(California).

2. Hạn chế

Mặc dù hệ thống hoạt động ổn định về mặt chức năng, nhưng vẫn tồn tại một số hạn chế do điều kiện triển khai và phạm vi đề tài:

- **Hạ tầng phần cứng:** Hệ thống hiện tại chạy trên môi trường giả lập (WSL/Virtual Machine) với tài nguyên giới hạn (RAM 8GB-16GB), chưa phải là cụm server vật lý chuyên dụng hay Cloud Cluster, dẫn đến khả năng chịu tải chưa thể đáp ứng quy mô dữ liệu doanh nghiệp thực tế.
- **Dữ liệu đầu vào:** Dữ liệu giao dịch hiện tại được mô phỏng từ tập tin CSV có sẵn và phát lại qua Kafka, chưa phải là luồng dữ liệu sống (live stream) kết nối trực tiếp từ các cổng thanh toán thực tế.
- **Phụ thuộc bên thứ ba:** Module lấy tỷ giá phụ thuộc vào API và website của Vietcombank. Nếu cấu trúc website thay đổi hoặc bị chặn IP do gửi request quá nhiều (dù đã có cơ chế delay 5 phút), luồng cập nhật tỷ giá có thể bị gián đoạn.
- **Độ trễ mạng VPN:** Do các node giao tiếp qua mạng Tailscale VPN, tốc độ truyền tải dữ liệu giữa các node (ví dụ từ Spark Worker sang HDFS DataNode) có thể bị ảnh hưởng bởi băng thông internet cá nhân, không nhanh bằng mạng LAN nội bộ trong Data Center.

3. Hướng phát triển

Để nâng cao hiệu quả và tính ứng dụng thực tế của hệ thống, các hướng phát triển tiếp theo được đề xuất như sau:

- **Tích hợp Machine Learning/AI:** Thay vì chỉ đánh dấu gian lận dựa trên nhãn có sẵn, hệ thống có thể tích hợp thư viện Spark MLlib để huấn luyện mô hình dự đoán gian lận (Fraud Prediction) theo thời gian thực dựa trên hành vi giao dịch.
- **Cơ chế phản hồi tự động:** Phát triển thêm tính năng gửi cảnh báo tức thì (Alerting) qua Email/SMS hoặc tự động khóa giao dịch khi phát hiện các dấu hiệu bất thường (ví dụ: chuỗi giao dịch cách nhau đúng 300s của Bot) thay vì chỉ hiển thị trên báo cáo.
- **Chuyển đổi lên Cloud:** Di chuyển hạ tầng từ On-premise Mini-Cluster lên các nền

tảng đám mây như AWS EMR hoặc Azure Synapse Analytics để tận dụng khả năng mở rộng (Auto-scaling) và bảo mật tốt hơn.

- **Tối ưu hóa bảo mật:** Áp dụng xác thực OAuth2 chặt chẽ hơn cho toàn bộ các service và mã hóa dữ liệu nhạy cảm (PII) của người dùng trước khi lưu xuống HDFS.

VII. TÀI LIỆU THAM KHẢO

[1] P. B. X. Q. E. M. a. S. B. Raghu Kuppala, "Write queries faster with Amazon Q generative SQL for Amazon Redshift," 07 NOV 2024. [Online]. Available: https://aws.amazon.com/blogs/big-data/write-queries-faster-with-amazon-q-generative-sql-for-amazon-redshift/?utm_source=chatgpt.com.

[2] Geeks For Geeks, "Generative AI vs. Discriminative AI," 07 Oct 2025. [Online]. Available: https://www.geeksforgeeks.org/data-science/generative-ai-vs-discriminative-ai/.

[3] AWS, "What is Generative AI?," [Online]. Available: https://aws.amazon.com/what-is/generative-ai/.

[4] "Apache Airflow," Apache, 2024. [Online]. Available: https://airflow.apache.org/docs/.

[5] "What is an on-premises data gateway?," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/power-bi/connect-data/service-gateway-onprem.

[6] "Power BI REST APIs - Datasets Refresh," Microsoft, [Online]. Available: https://learn.microsoft.com/en-us/rest/api/power-bi/datasets/refresh-dataset.

[7] "Databricks ODBC and JDBC Drivers," Databricks, [Online]. Available: https://docs.databricks.com/aws/en/integrations/jdbc-odbc-bi.

[8] D. w. N. t. Greek, "Small File Problems in Spark? Find out how to deal with it here - Load Big Data Efficiently (Part 7)," 2025. [Online]. Available: https://www.youtube.com/watch?v=jjWfSeSJWuU.

[9] Confluent, "Kafka Broker and Controller Configuration Reference for Confluent Platform," [Online]. Available: https://docs.confluent.io/platform/current/installation/configuration/broker-configs.html.

[10] Ming3993, "Introduction-to-Big-Data: Lab 1 Report," [Online]. Available: https://github.com/Ming3993/Introduction-to-Big-Data/blob/main/Lab%201/22120210/docs/Report.pdf..

[11] "What is Tailscale SSH? | Tailscale Explained," Tailscale, [Online]. Available: https://www.youtube.com/watch?v=08clF9srJ2k.